THE FRAMEWORK

Forensic page-build map, and how it differs from the bakeoff

A step-by-step, who-runs-what walkthrough of the top 6 page types you will build, which stages are run by Replit (deterministic code) versus an external LLM (your own API keys), what runs in sequence versus in parallel, and a side-by-side against the STS/DCD bakeoff. Worked example: MicroAnt, city of Naperville, service Data Destruction.

The one truth to anchor everything

Replit is the director, not the doer. Replit (platform code) runs orchestration and every deterministic step. The heavy generative work runs on EXTERNAL LLMs (Anthropic Sonnet and Opus, Google Gemini) billed to YOUR keys, never Replit's AI and never Replit tokens. The Replit chat agent is never in the runtime page loop at all.

SolveMy.AI internal reference. Generated 2026-06-28.

1. The engines (who actually runs each step)

Your framework ships exactly 6 canonical engines, locked by engines.lock.json (name + version + kind + sha256). Five are generative (a master prompt run verbatim on an external LLM). One, schema, is deterministic code with no model at all. This is the single most important difference from the bakeoff, so it sits first.

Engine	Kind	Runner / model	What it does
Master Page Builder	generative	External LLM: Sonnet 4.6 (write)	Builds HUB and SPOKE pages from the master prompt, fact layer, brief, link index, live SERP intel.
GEO Authority Builder	generative	External LLM: Sonnet 4.6 (write)	Builds NATIONAL authority pages (HQ-CID scope, long-form regulatory content).
Master SEO Audit	generative	External LLM: Opus 4.8 (reason)	Owns the research brief. Reads the full built page, does web + competitor research, returns findings (not HTML).
Hub + Spoke Planner	generative	External LLM: Opus 4.8 (reason)	Recommends the page plan across 5 tiers, exported as a CSV you import. This is where 'how many pages' is decided.
Schema Generator	DETERMINISTIC	Replit code (no model)	buildSchemaGraph emits one connected JSON-LD @graph. The .md is a spec only. No LLM, no Grok.
SEO Verify	generative	External LLM: Opus 4.8 (reason)	Forcing mini-prompt re-checks content + schema and must emit a PASS/FAIL verdict. Then the deterministic gate makes the final call.

Failover is live: if the main model fails or returns unusable output on a generative stage, a backup model takes over automatically and the swap is recorded in provenance. A genuine content FAIL is not a model failure and does not trigger failover.

2. The universal pipeline (sequence vs parallel)

Every engine-driven page runs the same 5-stage chain. Within one page the stages are STRICTLY SEQUENTIAL because each one gates the next: the audit brief gates schema, schema gates verify. Nothing inside a single page runs simultaneously.

#	Stage	Runner	Sequence note
1	Classify	Replit code (deterministic)	Resolves HUB/SPOKE/NATIONAL, checks engines.lock, confirms fact layer. No LLM.
2	Build	External LLM: Sonnet 4.6	Needs classify done. Live SERP intel is fetched here (SerpAPI required + DataForSEO) and cached per market.
3	SEO Audit	External LLM: Opus 4.8	Needs the built page. Reuses the cached SERP intel. Produces the research brief.
4	Schema + media	Replit code + Gemini (image)	Hard-gated: brief must exist AND (own CID OR explicit noGmb), else HALT. Schema is code; hero image is Gemini.
5	Verify + Gate	External LLM: Opus 4.8, then Replit code	seo-verify forcing check, then the Framework Enforcement Gate as the final blocker.

Across pages it is different: a hub + spoke batch FANS OUT, so many pages can run their own 5-stage chain IN PARALLEL. Parallel builds never clobber each other because the link index, manifest and sitemap merge through one shared-state source.

The 'execution prompt in between' you asked about is the execution preamble (the forcing layer, version-locked). It is not a separate turn: it is a system-message wrapper prepended to every generative call (build, audit, verify). It bans narration, declares the pinned model, and bridges the fact layer. There is NO chat-style intro handshake in our framework.

3. The top 6 page types you will come across

Important nuance so you are not oversimplifying: the engine has only 3 archetypes (HUB, SPOKE, NATIONAL) plus a non-engine utility page. The 6 'page types' below are surfaces you will name day to day; several of them map to the same SPOKE builder with a different module emphasis. The table says exactly which engine and what is skipped.

Page type	Engine archetype	Built by	Schema	Key modules	Skips
Home / Top Hub	HUB (page-builder)	Sonnet 4.6	Code: Org + WebSite + routing	Link-out routing to hubs + services	Deep module trio
City Hub (Naperville)	HUB (page-builder)	Sonnet 4.6	Code: service-area, HQ phone + areaServed, NO CID (noGmb)	Routing to that city's services, local proof	Deep module trio, own CID
Service / Spoke (Data Destruction)	SPOKE (page-builder)	Sonnet 4.6	Code: Service + LocalBusiness, most-specific type	Trio: grid.equipment + content.richtext + content.answerblock	(nothing material)
National Authority	NATIONAL (geo-authority)	Sonnet 4.6	Code: HQ-CID scope, regulatory entities	Long-form authority, CFR citations capped 4-8	City no-GMB handling
Equipment / Vertical (deep)	SPOKE (page-builder)	Sonnet 4.6	Code: Service/Product + LocalBusiness	Trio with equipment/ vertical emphasis	(nothing material)
Contact / Utility	none (no engine)	Static template	Optional ContactPage only	Contact-form module + Resend lead path	Audit, research brief, @graph, verify, SEO/ Lighthouse gates

Audit (Opus) and the gate (code) enforce the page-type rules. Examples it verifies: on the City Hub, no inherited HQ CID and correct areaServed; on the Service page, the module trio is present, CFR citations are capped 4 to 8 (never all 20), and compliance language is 'helps meet' not 'guarantees HIPAA'; on every page, favicon present, no em-dashes, no logos/text in AI imagery, redirects preserved.

4. MicroAnt builds a page now, vs you giving the bakeoff a URL

A) You ask MicroAnt (newest framework) to build the Naperville page

- **1. You request:** Pick client MicroAnt + page type + city/service in the dashboard, or it is already a row in the imported hub+spoke plan. Runner: you.
- **2. Classify + load:** Replit code resolves the archetype, checks secrets, loads the fact layer (Chicago HQ CID, Naperville noGmb). No LLM.
- **3. Live intel:** Replit calls SerpAPI (required) + DataForSEO and caches per market. External data APIs, not an LLM.
- **4. Build:** A background job calls YOUR keys: Sonnet 4.6 builds the page, wrapped in the forcing preamble. External LLM.
- **5. Audit:** Opus 4.8 reads the full HTML, does research, returns the brief (not HTML). External LLM.
- **6. Schema + image:** Replit code builds the @graph deterministically (Naperville: HQ phone + areaServed, no CID); Gemini makes the hero image.
- **7. Verify + gate:** Opus runs seo-verify, then Replit code runs the gate as the final blocker. If green, publish; memory + ledger logged.

Net: Replit directs and runs every deterministic step; Sonnet, Opus and Gemini are the doers on your keys. No human turns, no chat handshake, an automated gated chain, and multiple pages can run in parallel.

B) You give the bakeoff a URL for DCD / STS

- **1. You hand a target:** A URL or page spec into Mission Control. Runner: you.
- **2. Chat-style multi-turn run:** On external models: intro handshake, then page-builder turn (full HTML inline, narration-guarded), then seo-audit turn, then a Grok schema turn, then a seo-verify corrective loop (rebuild cap 2, fail-loud if it never hits an explicit PASS).
- **3. Live intel:** SerpAPI + DataForSEO fetched FRESH each build, no cache.
- **4. Design gate:** Rebuild cap, fails loud rather than shipping a drifted page.
- **5. Publish + QC:** You manually publish to Joomla; on confirmation, an automatic background QC fires (GTmetrix + JSON-LD checks).

Net: same DNA (masters run as a live pipeline on external LLMs), but the bakeoff is chat-shaped, single-tenant STS/DCD hardcoded, schema by Grok, no provenance lock, fresh SERP every run, manual publish then automatic QC.

5. Same as the bakeoff, and different from it

Shared DNA (both systems)

- Both run the real VA master prompts as a live pipeline, not canned output.
- Both do the heavy lifting on external LLMs billed to the operator, not on host/Replit compute.
- Both use live SERP intel: SerpAPI required (fail-loud) + DataForSEO best-effort.
- Both have a forcing / neutralizer layer plus refusal and narration guards that fail loud.
- Both have a design gate with a rebuild cap that fails loud instead of shipping drift.
- Both are converging: the bakeoff is adopting our deterministic schema, our engines.lock provenance pin, and our fact layer.

Where they differ (ours vs bakeoff)

Dimension	THE FRAMEWORK (ours)	Bakeoff (mc-core)
Schema	Deterministic code library, one connected @graph, no model	Production engine still has a Grok / AI schema turn (migrating to ours)
Tenancy	Multi-tenant fact layer per client, drift + bump	Single-tenant STS hardcoded (GMB now a fact object)
Provenance	engines.lock.json, gate fails on prompt drift	Vendored prompts, no provenance pin (drift undetected; adopting ours)
Enforcement	Installed distributable gate: runs in build + pre-commit hook, portable	Inline per-turn guards + validators, not an installed portable gate
SERP intel	Cached per market (DB-backed)	Fetched fresh every build, no TTL (a cost lever they want)
Orchestration	Background job queue, 5-stage gated, hub+spoke fan-out parallel	Chat-style multi-turn run with an intro handshake
Audit output	Returns findings; schema rebuilt by code	Corrective loop can rewrite full HTML inline
Publish + QC	Deploy; post-publish QC NOT wired yet (the gap to adopt)	Manual Joomla publish, then automatic background QC (GTmetrix + JSON-LD)
Cross-check	Grok dropped to a separate reviewer panel only	Reviewer / model cross-check in the loop

6. Where the mental model usually slips

- **Schema is not an LLM:** In our system schema is code, never Grok. That is exactly why the Naperville CID fix is enforceable: you cannot gate-prove a rule against a model's free output.
- **The audit does not hand back HTML:** Build (Sonnet) writes the page; audit (Opus) only returns findings; code rebuilds the schema. The model can suggest, it can never silently rewrite your schema, CID or locked facts.
- **There is no chat / intro handshake:** Ours is a background job. The 'execution prompt' is a preamble wrapper on every generative call, not a turn.
- **'Saving Replit tokens' is a non-issue:** Page builds never spend Replit tokens. They spend your external keys (Sonnet, Opus, Gemini). Model choice is a quality and per-provider-cost decision, nothing to do with Replit.
- **Two senses of 'Replit':** Replit the platform code orchestrates and runs deterministic steps. The Replit chat agent is never in the runtime page loop.
- **Page count is planned, not guessed mid-build:** The Hub + Spoke Planner (Opus) lays out the 5-tier plan as a CSV you import; the engine then fans the 5-stage build across the planned pages.

The two things genuinely worth taking from the bakeoff: post-publish live QC (GTmetrix + JSON-LD on the live URL) and a two-model audit cross-check. Everything else, the bakeoff is moving toward us.

7. The step-by-step sheet (worked example)

The whole pipeline as one grid: every step is a row, the three page columns show how each page type handles that step, and the Step column names who runs it (Replit code, or which external LLM). Assumptions locked so the columns are concrete: City = the Naperville HUB (a no-GMB service-area city: HQ phone + areaServed, no CID); Service = Data Destruction as a SPOKE deep page; Contact = a utility page that skips most generative steps on purpose. n/a = does not apply; Skipped = deliberately not run for that page type.

# / Step (runner)	City page (HUB, Naperville)	Service page (SPOKE, Data Destruction)	Contact page (utility)
1. You request the page (you)	Pick MicroAnt + Naperville + HUB	Pick Data Destruction + SPOKE	Pick contact, utility
2. Kickoff secrets check (Replit code)	Verify Anthropic + OpenAI + Gemini + SerpAPI + Resend keys; STOP loud if any missing	same	Only Resend (lead path) is load-bearing
3. Contact-form posture, step zero (you)	n/a	n/a	Keep existing form or build new; Resend notify posture set, never defaulted
4. Hub + Spoke Planner: page count (Opus, CSV import)	Recommends a 5-tier plan; Naperville is one HUB node in it	Recommends service spokes per priority city	Excluded, not a ranking page
5. Fact layer load (Replit code)	Loads Chicago HQ CID, Naperville noGmb = true	Loads services + compliance facts	Loads NAP + lead-notify target only
6. Design frame + modules, canvas skin (frozen skin)	HUB: routing / link-out modules to the city's services	Deep trio: grid.equipment + content.richtext + content.answerblock	Contact-form module only (skin-conformance gate deferred)
7. Stage 1 Classify (Replit code, no LLM)	Resolve HUB; assert engines.lock; confirm fact layer	Resolve SPOKE	Routed as utility, not through the engine
8. Live SERP intel (SerpAPI + DataForSEO, cached)	Pulls Naperville SERP now; cached for the audit to reuse	Pulls data-destruction SERP / PAA / competitors	Skipped
9. Execution preamble wrap (forcing layer, not a turn)	Wraps every generative call below: bans narration, declares the pinned model. No intro handshake	same	n/a
10. Stage 2 Build (external LLM: Sonnet 4.6, +failover)	page-builder (HUB) into the skin: facts + brief + link index + SERP intel	page-builder (SPOKE): emits the module trio	No engine build; static contact template
11. Stage 3 SEO Audit + brief (external LLM: Opus 4.8)	Sees full HTML; verifies no inherited HQ CID, areaServed, internal links, meta; returns brief, not HTML	Verifies trio present, CFR capped 4 to 8, compliance language ('helps meet' not 'guarantees HIPAA'), entities	Skipped (no research brief)
12. Schema hard gates (Replit code)	HALT unless brief exists AND (own CID OR noGmb); passes via noGmb	HALT unless brief + CID / noGmb resolved	No @graph gate
13. Stage 4 Schema @graph (Replit code, no model, no Grok)	service-area LocalBusiness: HQ phone + areaServed, NO CID	Service + LocalBusiness, most-specific type first	Optional ContactPage only
14. Hero image (external: Gemini direct)	AI-only, zero logos / text; merged into manifest + sitemap	same	Usually skipped
15. Stage 5 SEO Verify (external LLM: Opus 4.8, +failover)	Re-reads content + schema; must emit PASS / FAIL verdict; HALT blocks	same	Skipped
16. Framework Enforcement Gate (Replit code, final blocker)	em-dash, favicon, crawlability, redirect, image-logo, claims, verified-facts, CID correctness	+ deep-modules, CFR / claims	favicon + em-dash + mobile only; SEO / Lighthouse exempt
17. Lead path + form-health (Resend + protected-settings)	n/a	n/a	Wire Resend handler; protected-settings fail-loud; recurring synthetic form-health armed
18. URL preservation / redirects (redirects.json 301/410)	Preserved on any rebuild / migration	same	same
19. Publish (deploy)	Dev and prod DBs are separate; verify live	same	same

# / Step (runner)	City page (HUB, Naperville)	Service page (SPOKE, Data Destruction)	Contact page (utility)
20. Post-publish QC (not wired; bakeoff has it)	Would adopt: live GTmetrix + JSON-LD	same	Lighter QC
21. Project Memory + ledger (Replit code + you)	Source artifacts captured; decision logged in full detail	same	same